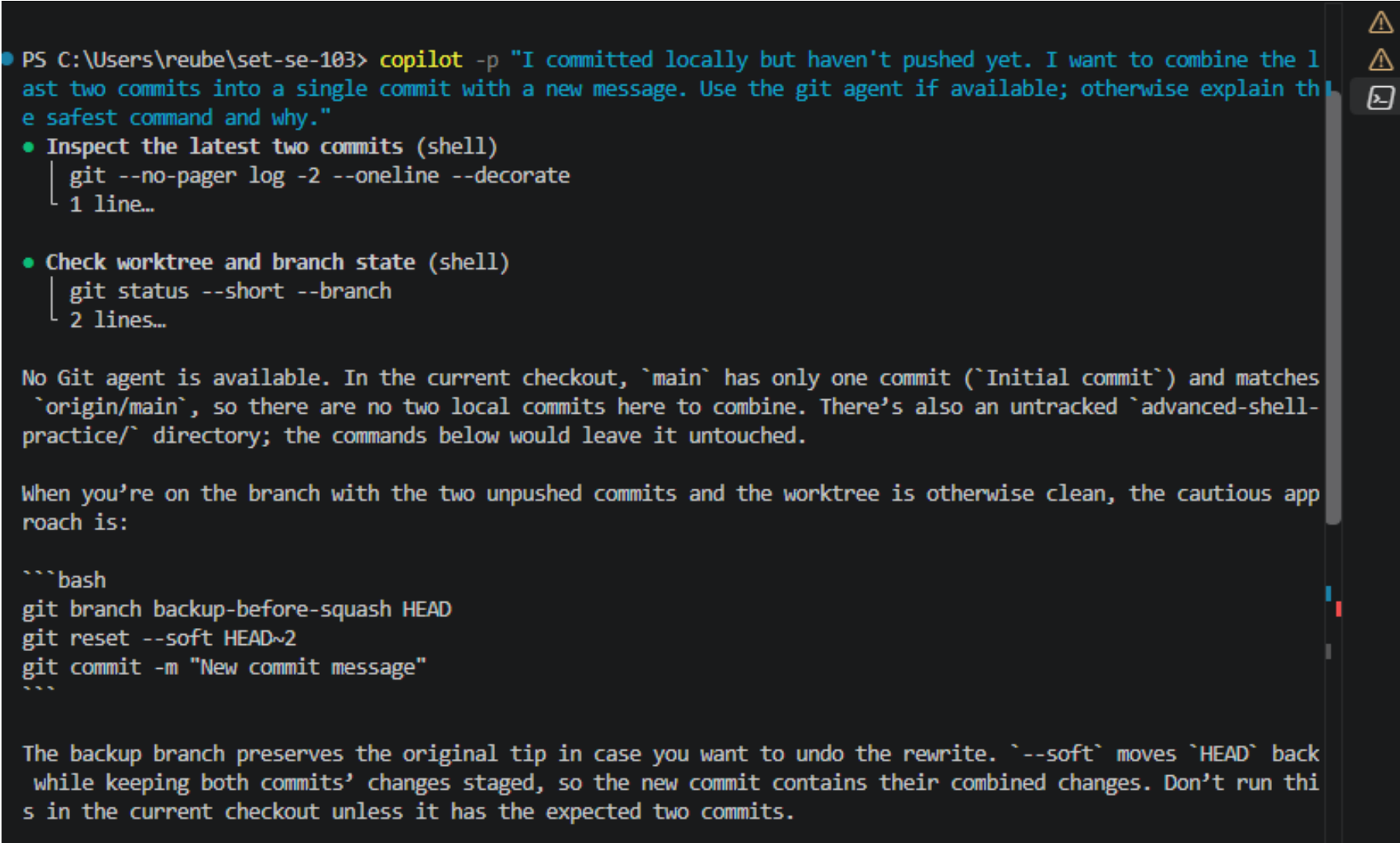


Reuben_AI: Mastering the Shell with GitHub Copilot CLI

Copilot CLI terminal screenshot



PS C:\Users\reube\set-se-103> **copilot** -p "I committed locally but haven't pushed yet. I want to combine the last two commits into a single commit with a new message. Use the git agent if available; otherwise explain the safest command and why."

- Inspect the latest two commits (shell)
 - | `git --no-pager log -2 --oneline --decorate`
 - | 1 line...
- Check worktree and branch state (shell)
 - | `git status --short --branch`
 - | 2 lines...

No Git agent is available. In the current checkout, `main` has only one commit (`Initial commit`) and matches `origin/main`, so there are no two local commits here to combine. There's also an untracked `advanced-shell-practice/` directory; the commands below would leave it untouched.

When you're on the branch with the two unpushed commits and the worktree is otherwise clean, the cautious approach is:

```
```bash
git branch backup-before-squash HEAD
git reset --soft HEAD~2
git commit -m "New commit message"
```
```

The backup branch preserves the original tip in case you want to undo the rewrite. `--soft` moves `HEAD` back while keeping both commits' changes staged, so the new commit contains their combined changes. Don't run this in the current checkout unless it has the expected two commits.

The screenshot shows the Copilot CLI response. Its lower edge is cropped, so the final command is not visible.

Git command and CLI follow-up

CLI prompt: "I committed locally but haven't pushed yet. I want to combine the last two commits into a single commit with a new message. Use the git agent if available; otherwise explain the safest command and why."

I also tried the assignment prompt with GitHub Copilot CLI in a terminal. The CLI reported that no git-named agent was available. It checked the current repository and found only one commit, so it warned that HEAD~2 could not be used safely yet.

Command for a branch with at least two local, unpushed commits

```
git reset --soft HEAD~2  
git commit -m "Your new commit message"
```

The soft reset moves the current branch back two commits while keeping the combined changes staged. The new commit then records those changes under the replacement message. Check the current branch, commit history, and working tree first; do not run this when the required commits are not present.

Accuracy ranking and justification

Rating: 8/10. The command is correct when the last two commits are local and unpushed, but the initial answer did not warn me to verify that condition or explain what changes if commits have already been pushed. Rewriting published history can disrupt collaborators and may require a carefully coordinated force-push, so it should not be done casually.

Comparison with git rebase -i

An interactive rebase, such as `git rebase -i HEAD~2`, lets me choose squash or fixup and edit the combined commit message in the sequence editor. It is more flexible when I need to reorder, drop, or edit commits; the soft-reset approach is shorter when I simply want to combine the last two commits and create one new message.